



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

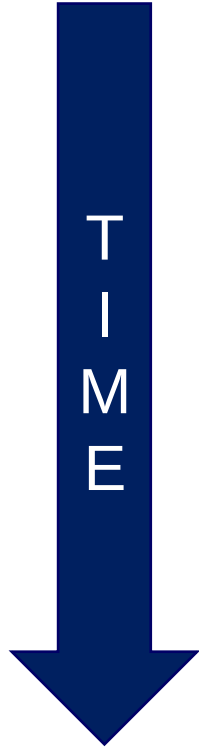
Modern software paradigms

Michele Colajanni

Dipartimento di Informatica – Scienza e Ingegneria
michele.colajanni@unibo.it

Ways to implement a service

1. Monolithic
2. Service Oriented Architecture (SOA)
3. Microservices
4. Serverless
5. Batch processing vs Real-time processing



The overall picture

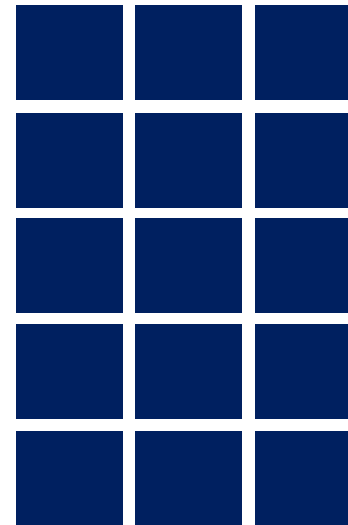
Monolithic



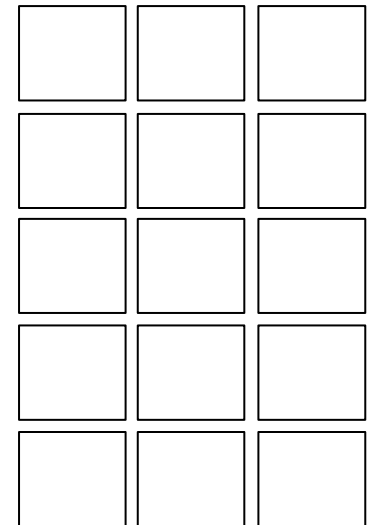
Service Oriented Architecture(SOA)



Microservices

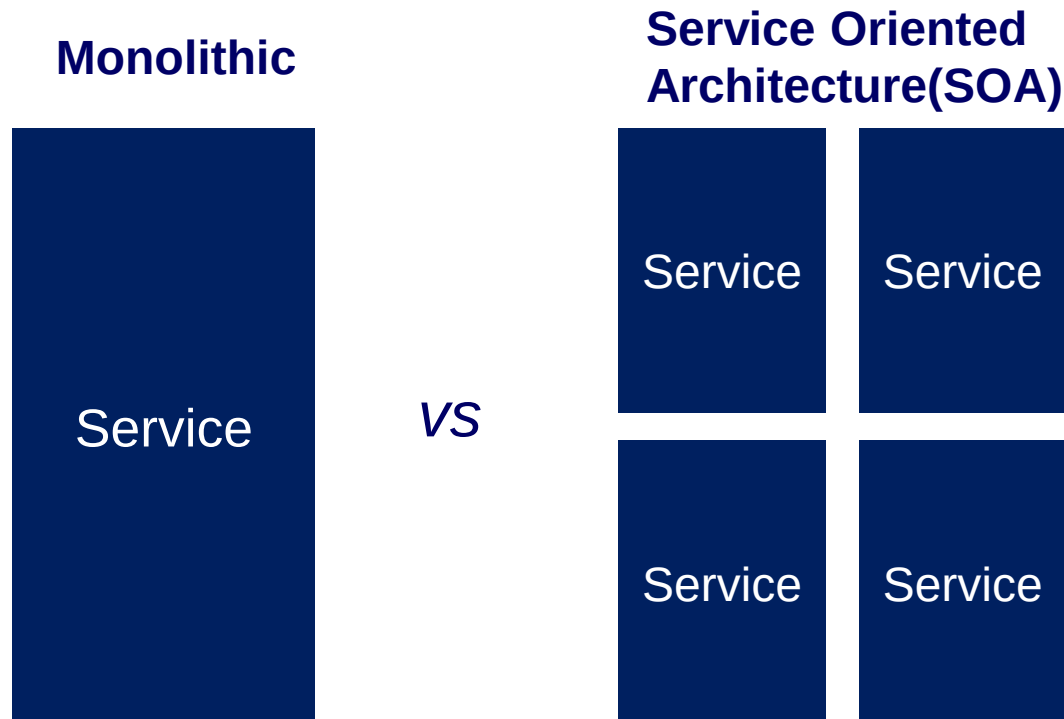


Serverless



SOA brought an initial partition

- Fine grained (with respect to monolithic applications), reusable, scalable services
- Coupled through orchestration and ESB
- For SOA, it is still common to use large servers and databases



SOA and ESB



Mechanisms offered by an Enterprise Service Bus (ESB)

ESB is a middleware supporting the infrastructure and tools to **host applications and services and facilitate their interactions** through several mechanisms, among which:

- Base routing messages to destination
- Managing different communication models and ways of interactions among services, such as asynchronous queuing, event-driven messaging, reliable messaging, service intermediation (broker)
- Connectors and bridges for different protocols transforming data and protocols between service requestor and service provider
- Handling business events and delivering them by providing policies and Quality of Service (QoS), and also mechanisms for monitoring, logging, and security



Modern services

Modern services are different from traditional applications because they have strong requirements (**scalability, flexibility, availability**) in large distributed infrastructures where many hardware, software and network problems can emerge

Nevertheless, a modern service must be:

- **resilient** enough to handle servers coming and going at will
- **ready for failure**
- **elastic**, that is, able to scale from few servers to tens of thousands of servers and back, hence →
- **agile**
- **loosely-coupled**
- as **stateless** as possible



From SOA to Microservices

Motivations for SOA

- Break the monolithic software
- Integrate legacy apps
- Scale and reuse

Motivations for Microservices

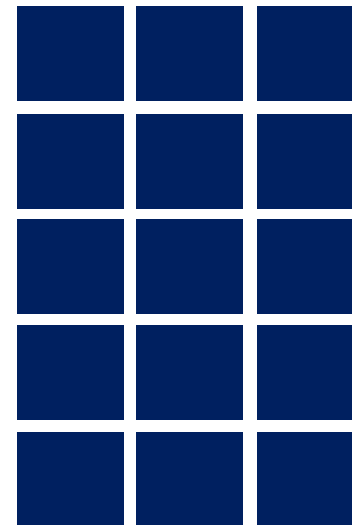
- Easier to implement and test
- Agility, elasticity, high reusability
- Not tied to servers, tools, DBs

Service Oriented Architecture (SOA)



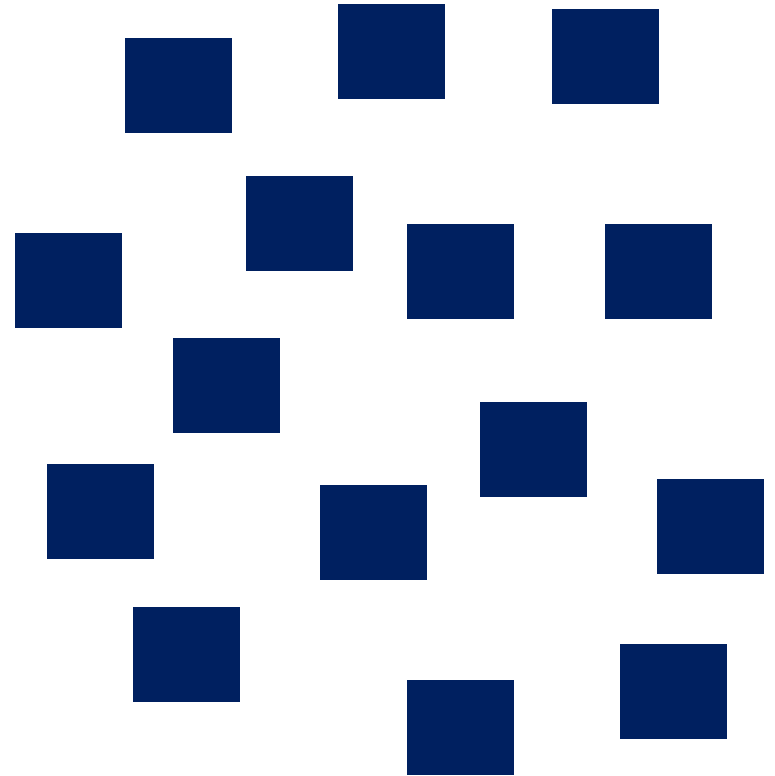
VS

Microservices



Distributed systems are *tough* (in any case)

- Despite all, **SOA** are rigid but they seem not too bad from the external because they hide most of the internal complexity
- **Microservices** make explicitly visible several complexities and challenges:
 - **Communications**
 - **Coordination**
 - **Service discovery**



Microservices

The **microservice architectural style** is an approach to:

- developing a single application as a suite of very small business task implemented as services
 - each running in its own process
 - and communicating with through some lightweight mechanisms (often an HTTP resource API)
- These services are **built around business capabilities** and may be written in different programming languages and use different data storage technologies
 - There is the necessity of **orchestration** for these microservices that work well if they are independently deployable by fully automated mechanisms



Evolution: No one was born on a *microservice* basis

- **eBay** (5th generation in 2019)
Monolithic Perl → Monolithic C++ → SOA Java → **Microservices**
- **Twitter** (3rd generation in 2019)
Monolithic Ruby → SOA Java / Scala → **Microservices**
- **Amazon** (9th generation in 2019)
Monolithic Perl / C++ → SOA Java / Scala → **Microservices**

2020: All front-end (client-side) of the most popular Web-based services (Amazon, eBay, Facebook, Google, Linkedin, Twitter, Wikipedia, Yahoo, Youtube, etc.) are written in **JavaScript**



How to define a microservice

It is not about the dimension of the code, but it is a portion of code that is

- 1) single-purpose
- 2) well-defined and simple interface
- 3) modular
- 4) independent
- 5) isolated persistence → each microservice has to manage its storage data



Example of microservices

Service: Implement the entire software management of an online request for a car insurance

Example of microservices

1. Receive the client request
 2. Verify whether the client is an already registered customer
 3. Check the level of risk of a specific car
 4. Evaluate the insurance premium
 5. ...
- Offer each of them as a microservice
 - It is possible to use these basic services for creating more complex services



Similarities between SOA and Microservices

- **Both SOA and microservices allow a complex application to be broken down into smaller, more manageable, and somewhat independent pieces**
- Given the fact that dividing an application into smaller parts could be handled by either SOA or microservices, there are those who insist that there is not a real difference between the two architectures
- **In fact, there are actually some crucial differences between the two architectures, and these differences play into the ways in which the architectures should be used**



Differences between Microservices and SOA (1)

1. **SOA Web-based services are today considered as coarse grain services**, because they are developed and deployed as large units. In contrast, **microservices are developed independently**, and can even be built using different tools and different languages, so long as the microservices conform to the standards set forth by the API
2. **Microservices tend to be far more granular than SOA**. They do one thing, and do it well even because each of them is well tested
3. **Microservices also tend to be easier to test than SOA services**, because they only perform one function. They are designed for re-use and to work independently



Differences between Microservices and SOA (2)

4. Microservices are individual services that are able to **communicate with one another through APIs (and not through an ESB)**
5. Microservices are **truly independent of one another**. It might be possible, for example, to upgrade a microservice without upgrading any of the other application components. Similarly, it is possible to shut down microservices that are not currently being used, without impacting the other microservices that continue to run
6. Typically, there is a logical separation between microservices. **Individual microservices often run in (cloud-based) containers**



Benefits

- **Enables the continuous delivery and deployment of large, complex applications**
 - Better testability - services are smaller and faster to test
 - Better deployability - services can be deployed independently
 - It enables you to organize the development effort around multiple teams. Each team owns and is responsible for one or more services. Each team can develop, deploy and test their services independently
- **Each microservice is relatively small**
 - Easier for a developer to understand
 - The application starts faster, which makes developers more productive, and speeds up tests and deployments
- **Improve availability through fault isolation and replication**
 - For example, if there is a memory leak in one service then only that service will be affected



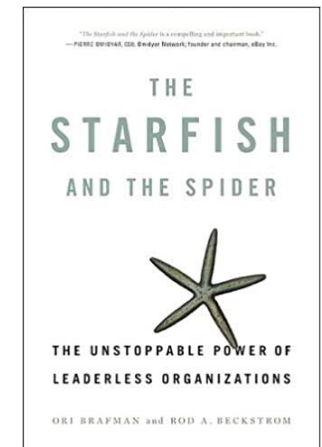
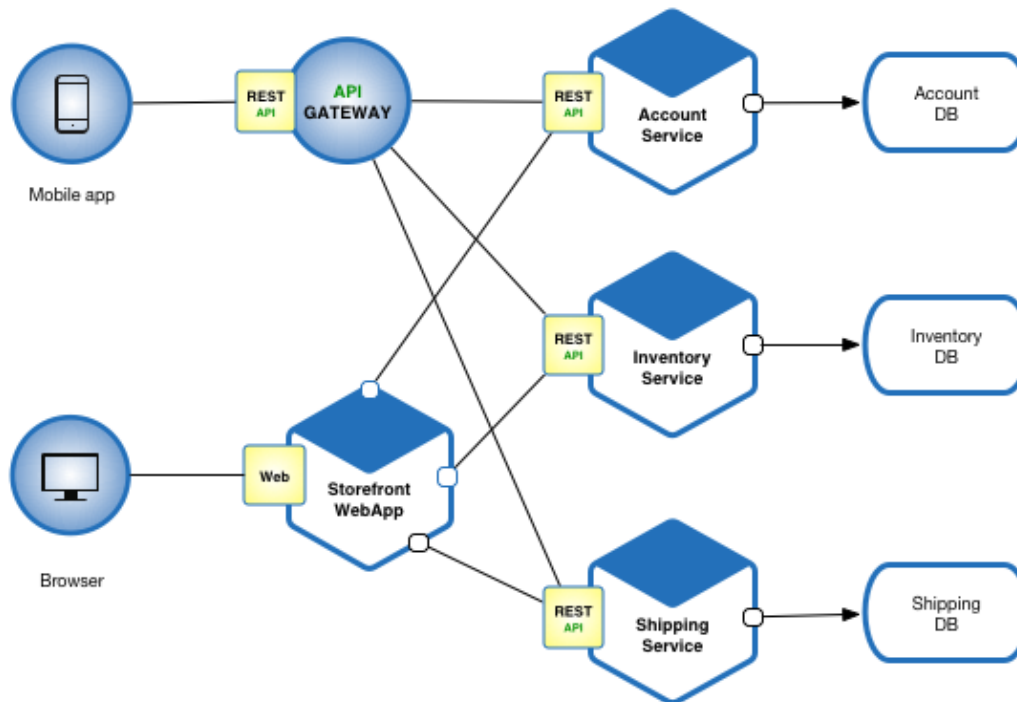
Drawbacks

- **Many developer tools** do not provide explicit support for implementing and testing several distributed microservices
- Lack of competences in the developer community
- Developers must implement
 - complex **inter-service communication mechanisms**
 - complex **orchestration mechanisms**: in production, there is the operational complexity of deploying, managing and monitoring a system consisting of many different services and service types
- **Increased process and memory consumption.** The microservice architecture replaces N monolithic application instances with $N \times M$ services instances. The overhead is higher



Diffusion of microservices

- Most large-scale Web-based services including **Netflix**, **Amazon** and **eBay** have evolved from a monolithic SOA architecture to a microservice architecture
- For example, the Amazon.com website application calls 100-150 services to get the data that used to build a web page



Many alternatives

Characteristics of a Microservice Architecture

- Componentization via Services
- Organized around Business Capabilities
- Products not Projects
- Smart endpoints and dumb pipes
- Decentralized Governance
- Decentralized Data Management
- Infrastructure Automation
- Design for failure
- Evolutionary Design

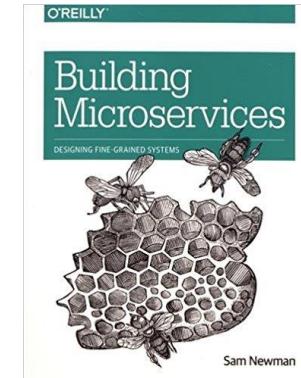
Sidebars

- How big is a microservice?
- Microservices and SOA
- Many languages, many options
- Battle-tested standards and enforced standards
- Make it easy to do the right thing
- The circuit breaker and production ready code
- Synchronous calls considered harmful

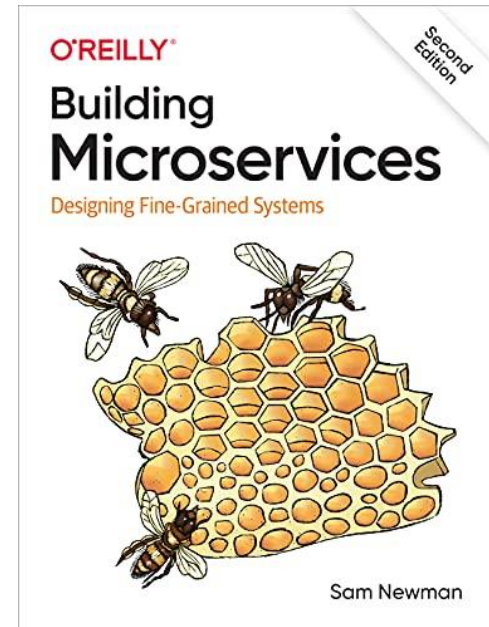


Reference book

**“Building Microservices”,
Sam Newman, 2015**



**“Building Microservices:
Designing Fine-Grained
Systems - 2nd Edition”,
Sam Newman, 2021**



The fundamental role of virtualization

It represents the first step to free *Computer science* from *Electronics*, and *Software* from *Hardware*. Long history:

- Operating System
- *Time sharing* Operating System (prototyped in Multics, it was implemented by IBM in late '60s – early '70 with IBM/360 and CP/CMS, IBM/370 and VM/370)
- Graphical user interfaces
- Virtual machines and different OS on same hardware (e.g., VMware)
- Hypervisor
- Software Defined Network
- Hyperconvergence
- Containers (e.g., Docker)
- Orchestration of containers (e.g., Kubernetes)
- ...
- Augmented reality, Virtual reality, E-gaming, ..., Metaverse



Dominating architectures

- In the middle '80s, **client/server** architectures
- In the middle '90s, **http client (browser)** and **server** architectures
- In the 2000s, **virtual machines** (VMs) created the ability to emulate multiple resources on one computer
- In the 2010s, **cloud** hits the top of virtualization by helping provider companies become more agile on development and focused on operations without caring of real computer and disk infrastructures
- In the 2020s, **containers, orchestrators, microservices** and **serverless technologies** are dominating the scene
- Virtualization is permeating even the traditional industrial world thanks to the virtualization of ERP, MES and SCADA servers



Virtualization

- The computational capacity of a single server became so large that it was impossible for most workloads to effectively use it
- Even though the technology and use cases have evolved, the core meaning of virtualization remains the same: **to enable a computing system to run multiple independent and heterogeneous applications at the same time**
- **Virtualization** replaces a physical system with a virtual version of some resources, e.g.,
 - operating systems
 - servers
 - storage devices
 - network connections and appliances



Virtualization and cloud computing

Virtualization is the most important enabler of cloud computing because it improves resource utilization and simplify datacenter management

- Datacenters use virtualization techniques to:
 - make abstraction of the physical hardware
 - create large aggregated pools of logical resources consisting of CPUs, memory, disks, file storage, applications, networking, security appliance, ...
 - offer those virtual resources to users or customers in the form of agile, scalable, consolidated virtual machines

No *elasticity* is possible without virtualization



Server virtualization

- **Server virtualization** masks server resources, including the number and identity of individual physical servers, processors, and operating systems, from users and applications
- The intention is to spare the user and application from having to understand and manage complicated details of server resources while
 - increasing resource sharing and utilization
 - improving load balancing and availability
 - maintaining the capacity to expand later



Two remarks

When a new technology appears, previous technologies survive (typically, for a long time).
Hybrid architectures prevail in most companies

When a new technology appears,
**you should NOT evaluate the pros and cons, and
you should NOT forecast the utilization models**
looking just at *present* models and workloads
→ *New offers create new demand and new service requirements*



Containers



Motivation for containers

- The *virtualization* is the first huge step but, as a unit of scalability, **the virtual machine is expensive**
 - Each VM includes all the complexity of every other operating system instance: a full copy of the operating system, device drivers, libraries, and other implementation-specific software
 - Containers are like a lightweight virtual machine that
 - has its own process space
 - has its own network interface
 - can run stuff as root
 - can install packages
 - can run services



Containers

- Containers are a standardized way to package an application, its configuration and dependencies into a single logical object. In practice, a container is a small software package which, ideally, performs a small, well-defined task. It appears as a set of visible processes on the host machine (note: VMs are opaque)
- Container images include all the software, including settings, libraries, and other dependencies, needed for them to run and can guarantee:
 - Standard packaging
 - Isolation and efficiency
 - Portability
 - Separation of concerns



Containers and cloud

- Containers are considered cloud-native because
 - they help **accelerate the development** and deployment processes
 - they promote a development methodology for applications and services that are intrinsically **scalable** and **reliable**
 - they are easy to manage and to **modify**, so they are better suited to frequent changes, for technical or business reasons
 - they make workloads **portable** and even mobile between different servers and clouds, and are the ideal basis from which to build *software-defined infrastructures*



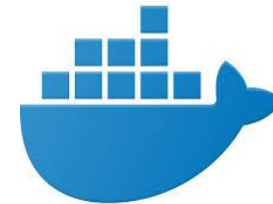
The role of the open source community

- **Containers represent the solution for something *simpler, slimmer, and more controllable* than a virtual machine**
- The software infrastructure for containers is primarily *open source*, written by highly talented developers with the support of many large groups (first of all, Google)
- Keys enabling features for containers:
 - ***namespaces***: (from Linux kernel v. 2.4.19, Aug. 2002) allows partitions of kernel resources such that a set of processes sees some resources and another set sees different resources
 - ***cgroup* (control group)**: (from Linux kernel v. 2.6.24, Jan. 2008); it is a feature of the Linux kernel that limits and isolates the usage of the resources (e.g., CPU, RAM, disk I/O, network I/O) of a set of processes

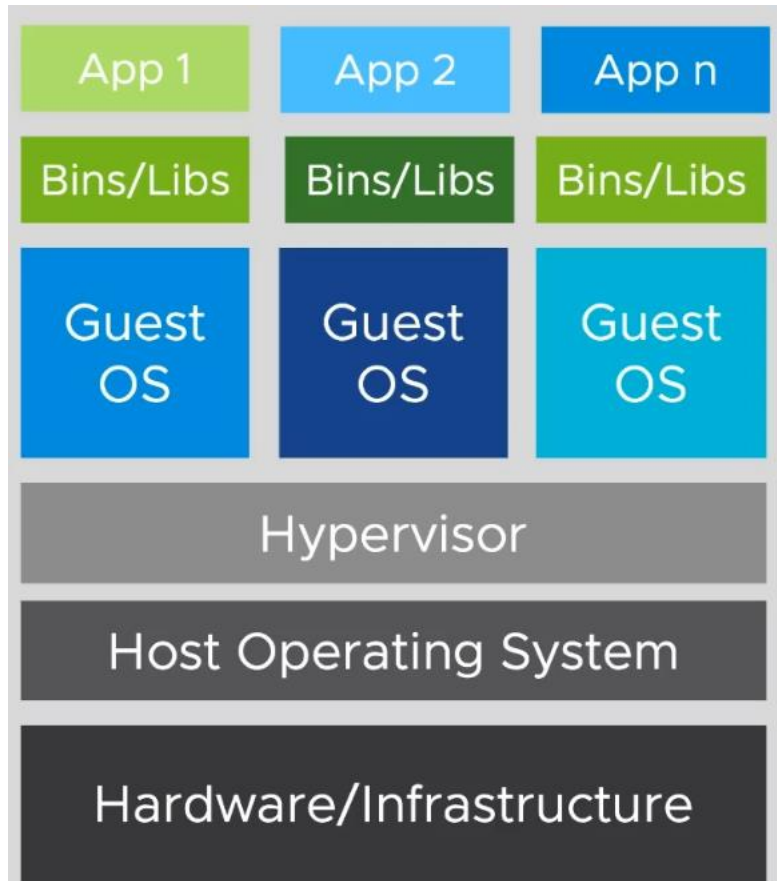


Docker

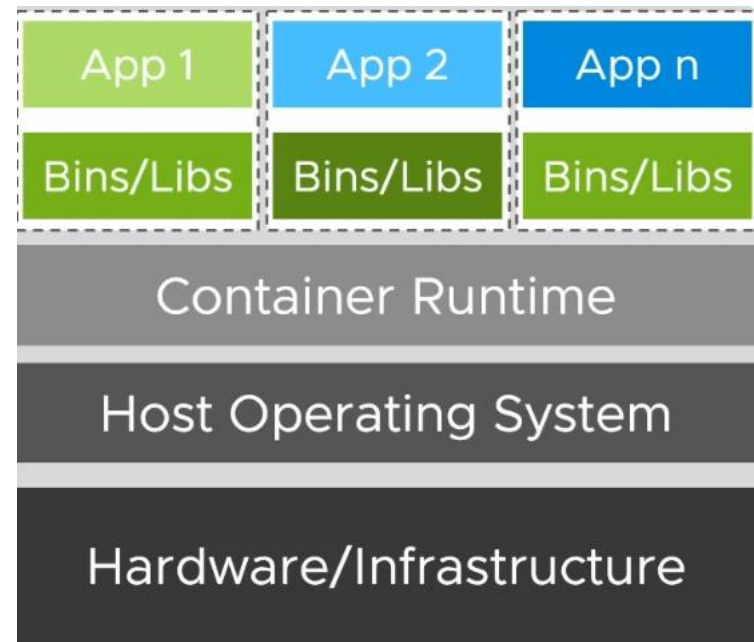
- Containers were popularized by **Docker**, the company which built the open source Docker distribution and many commercial products based on it
- There are other container implementations, but **the vast majority of container applications run on Docker and Linux**



VM vs. Containers



Virtual machines



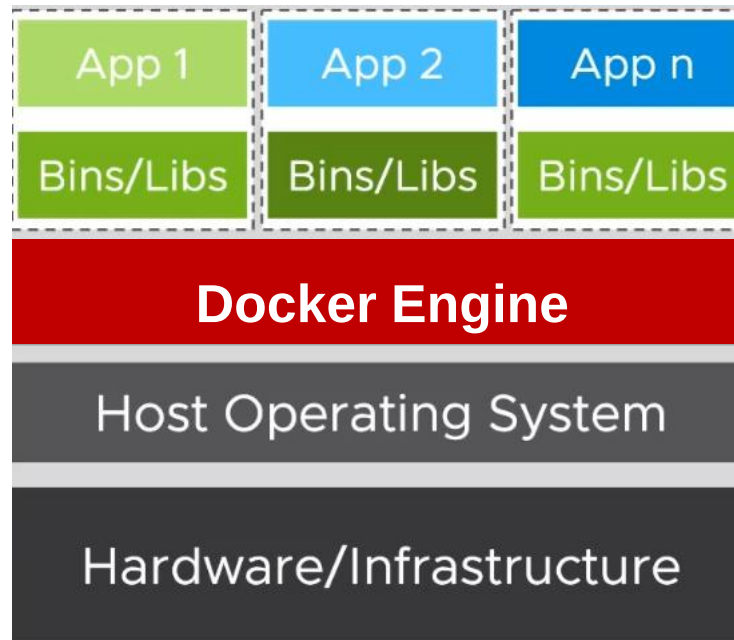
Containers



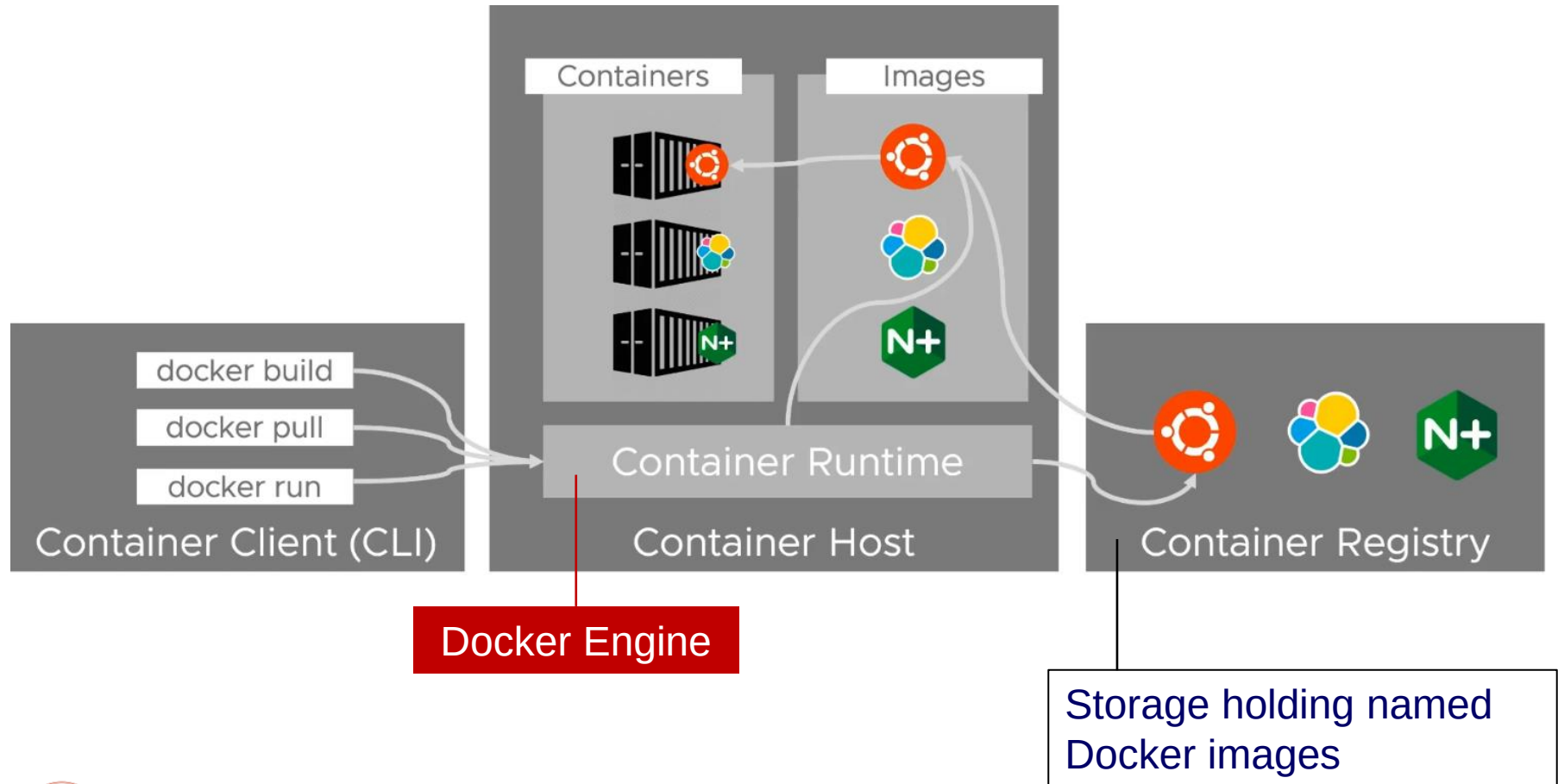
Containers at runtime

From a computer point of view, **containers** are a process that accesses a single kernel and is

- typically extracted from a *tar archive*
- anchored to the definition of its *namespaces*
- controlled by *cgroups*:



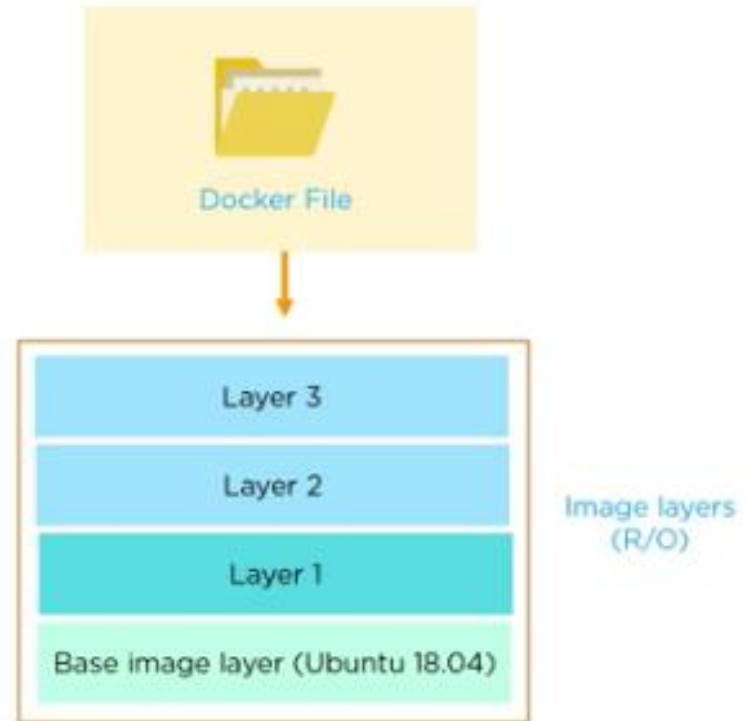
Containers at runtime



Docker image

Docker Image

- A Docker Image is a template of instructions which is used to create containers
- A Docker Image is built using a file called Docker File
- It is comprised of multiple layers
- By default, Docker Image starts with a base layer
- Here, each layer depends on the layer below it
- Image layers are created by executing each command in the Dockerfile and are in the read-only format

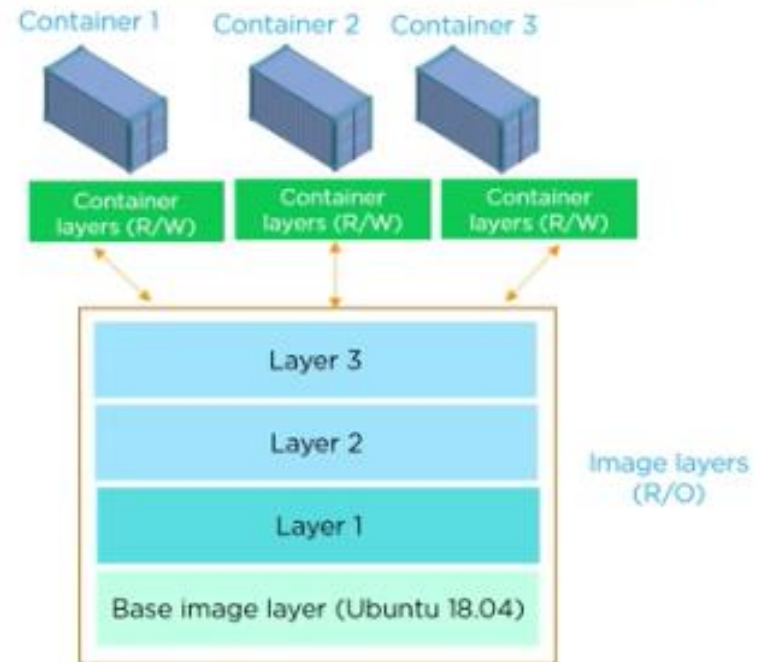


Docker image (2)

Docker Image

- Whenever a user creates a container, a new layer is formed on top of the image layers called container layer
- Every container has a separate (R/W) container layer and any modification in a container is reflected upon the container layer alone

Container based on the Ubuntu 18.04 image:



Docker Container

Docker Container

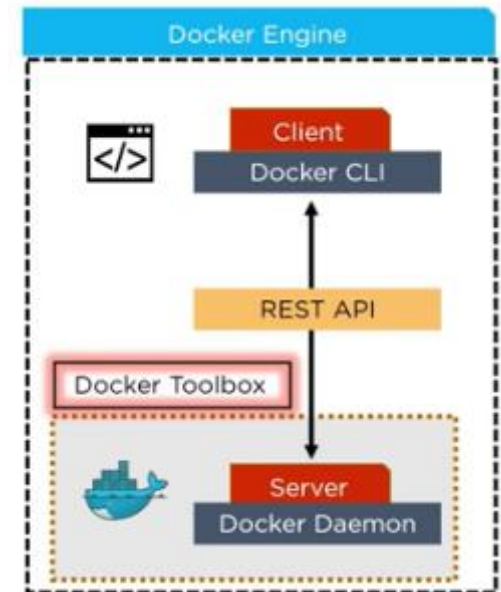
- Docker Container is an executable package of application and its dependencies together
- Since it's light-weight, it can be easily deployed and executed on other computer environments regardless of their host OS/ configurations
- Docker Containers run applications in isolation and also share the OS kernel with other containers



Application runs in isolation

Docker engine

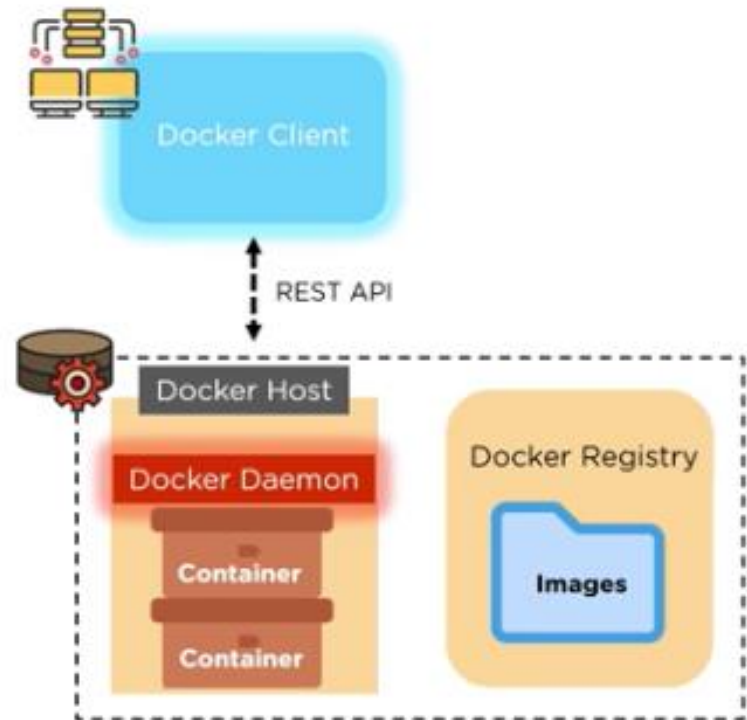
- Docker engine or Docker is a client/server application that builds and executes containers using Docker components
- REST API is a primary mode of communication between Docker Client and Docker Daemon



Docker client

Docker Client

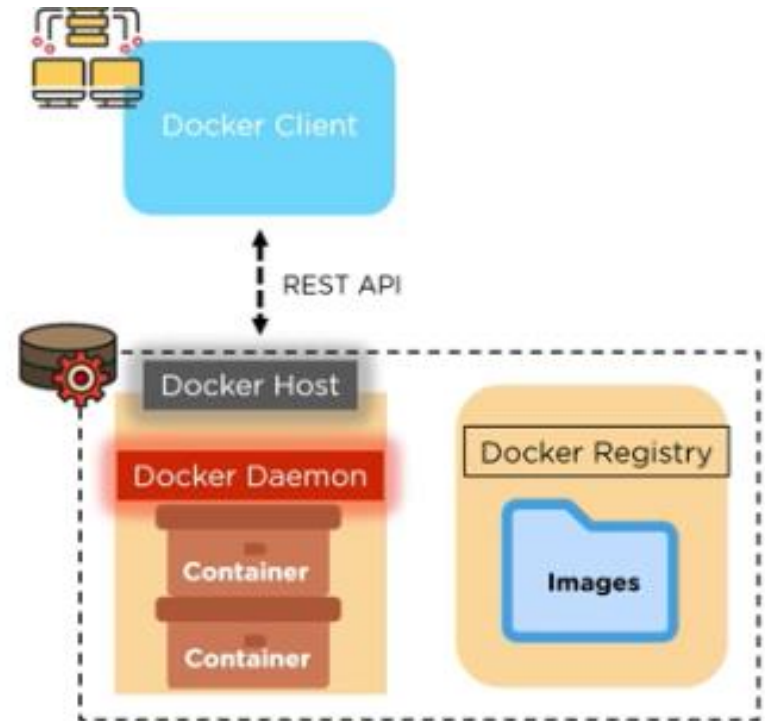
- Docker Client consist of the CLI command which is used to issue commands to the Docker Daemon
- Docker Client uses REST API to issue commands to Docker Daemon through scripting or direct CLI commands
- For example, when you use a *docker pull* command, the client sends this command to daemon, which performs the operation by interacting with other components (Image, Container, Registry)



Docker server

Docker Daemon (server)

- Docker Daemon is a server which interacts with the operating system and performs all kind of services
- The Docker Daemon listens for REST API request and performs the operation
- A command *dockerd* is used to start a Docker Daemon
- Docker Host runs the Docker Daemon and Registry



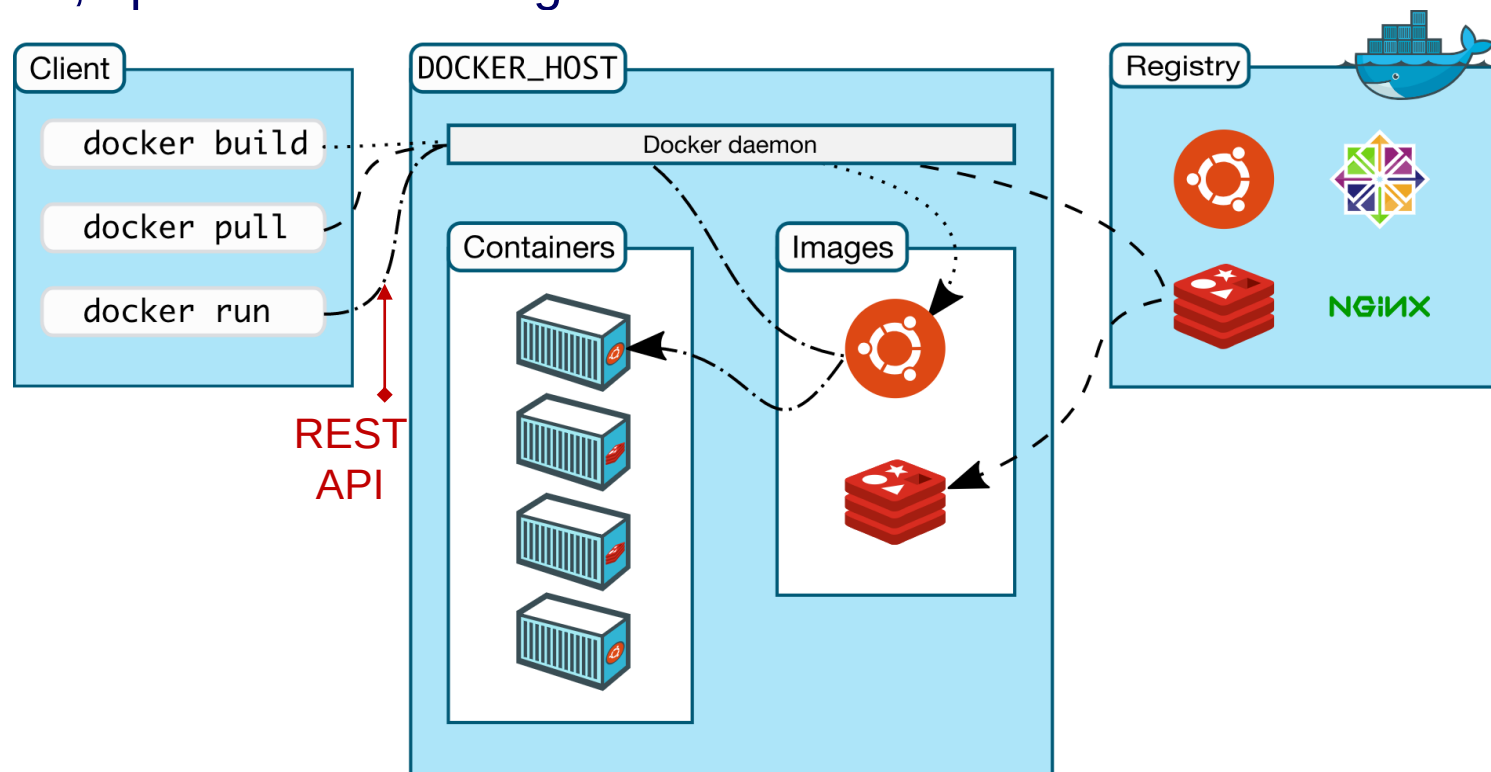
Runtime management

- A controller program, such as the **Docker Engine**, is necessary to manage the containers at runtime
 - Unlike virtual machines (VMs), containers can be created and destroyed quickly and with few resources. They include the software necessary for them and little else
 - Like VMs, containers are isolated from each other (at a lower degree)
 - More than VMs, containers are abstracted from the underlying infrastructure. This makes them portable across different servers, clouds, and potentially operating systems
- Because the developer packages the container with the application and all its dependencies, their own list of processes and file system space, it becomes an easily portable function package → different container applications can run dependent software which, running in the same instance, would conflict

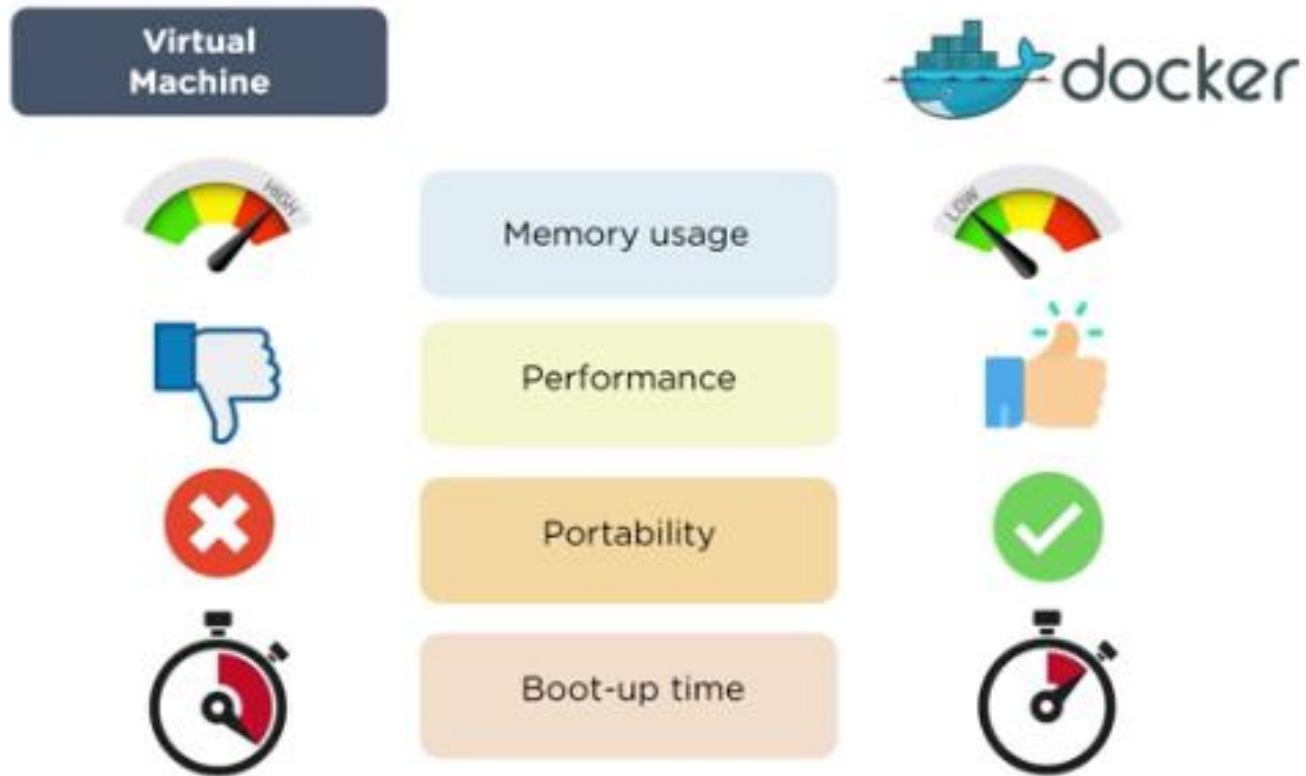


Docker is a client/server architecture

- **Docker Client** is the utility we use when we run a docker command. For example, `docker build`, `docker run` (docker container run), `docker images`, `docker ps`, etc.
- **Docker Daemon/Engine** is the part which really executes the command and knows how to talk to the OS kernel, makes the system calls to create, operate and manage containers

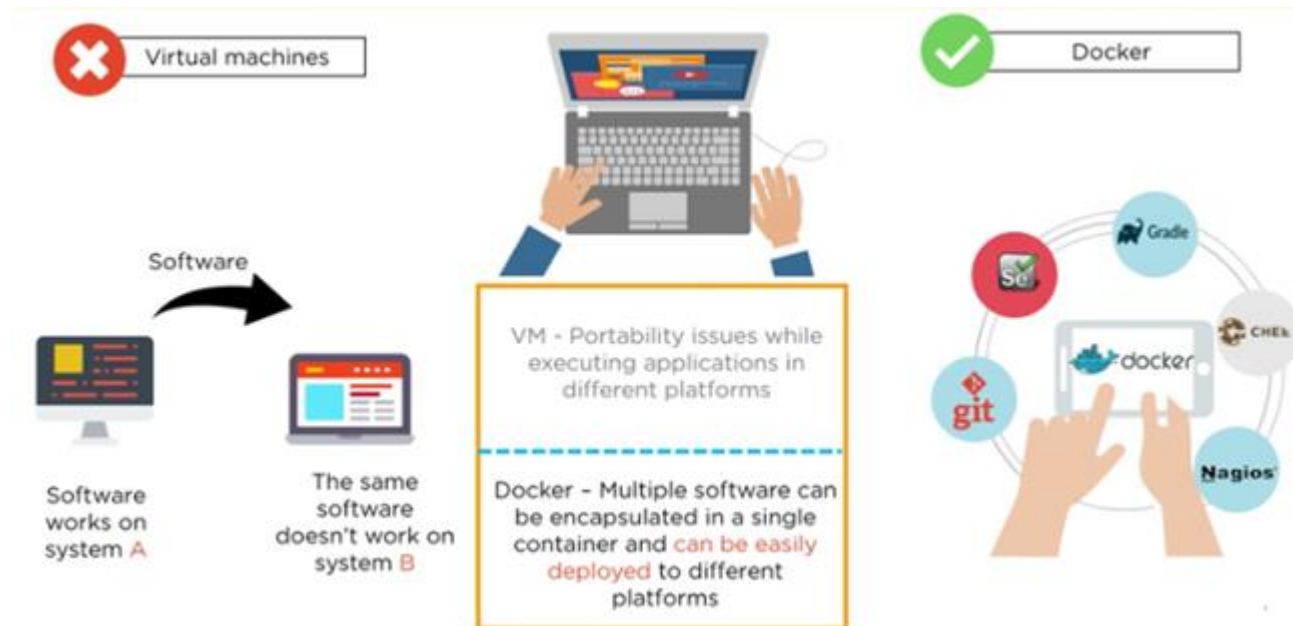


Virtual machines vs Containers



Portability

- Because application containers do not have the VM overheads, it is possible to support more containers on the same infrastructure
- **Portability** is one of the main benefits. As long as server settings are identical across systems, an application container can run on any system and in any cloud without requiring code changes. There are no guest OS environment variables or library dependencies to manage



Microservices and containers

- Microservices are container-based software built to provide a service consumed by other software, including other microservices
- They need not be but they are typically associated with:
 - Containers
 - DevOps
 - Orchestration
 - the rest of the cloud-native architectures and techniques
- As microservices are designed to perform simple, well-defined functions, rather than large collections of functions, they facilitate development by different teams and testing at all stages, and allow the system to scale only the services needed

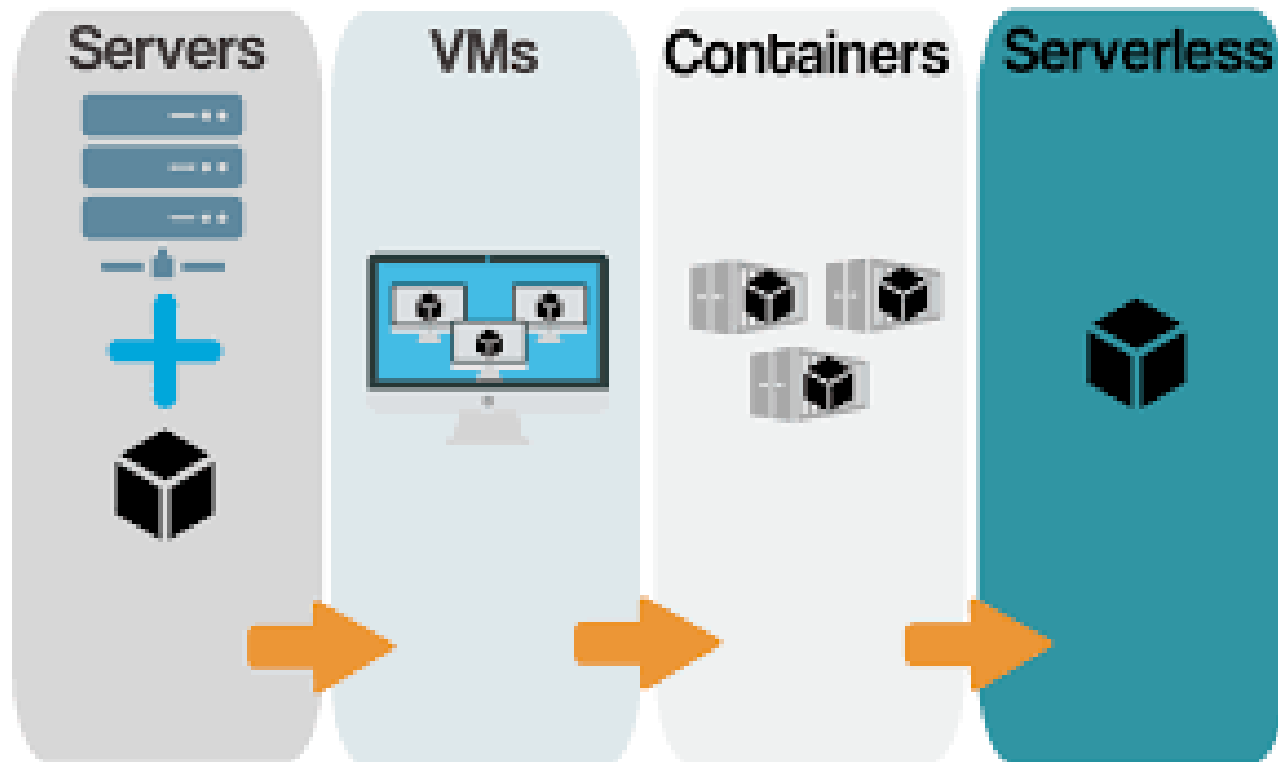


Microservices and Serverless

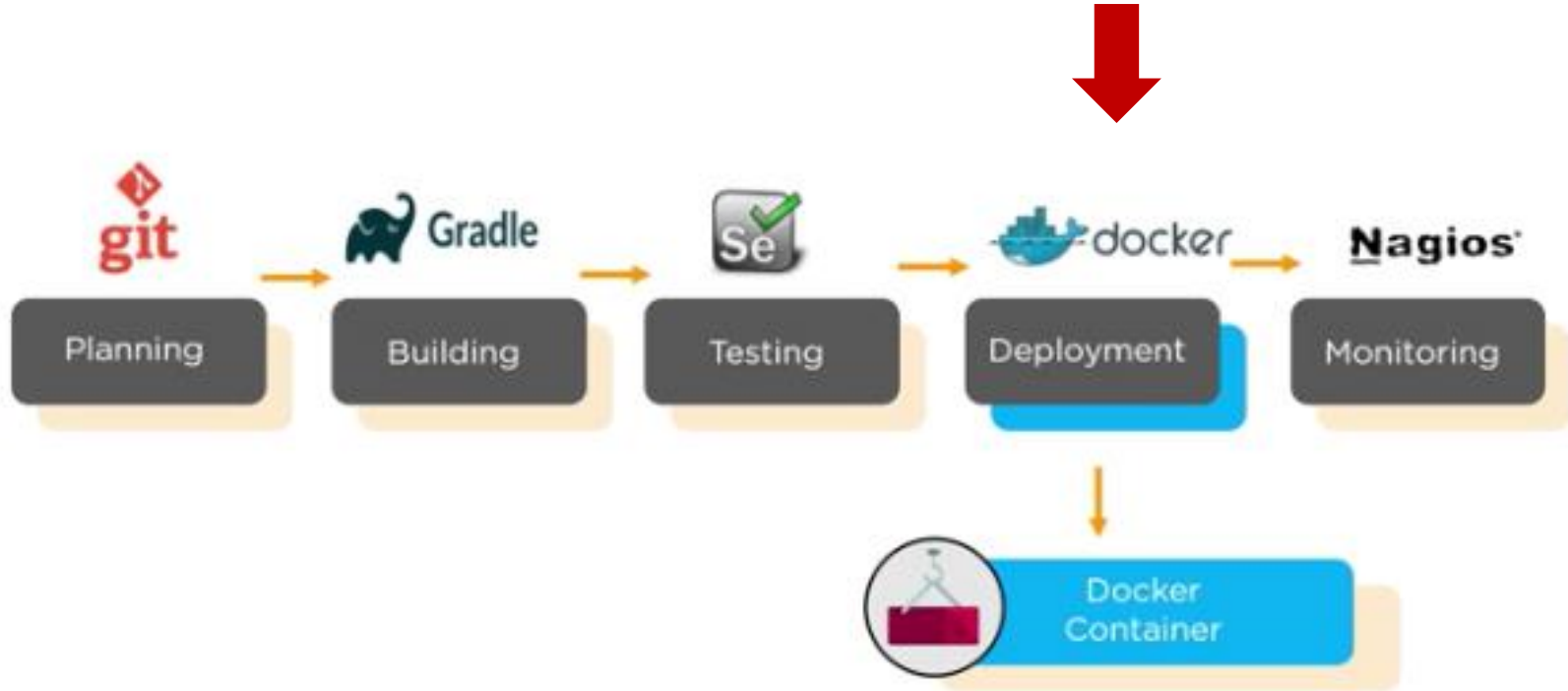
- **Microservices is a way to design an application**
- **Serverless is a way to run an application** (or a part of it)
- Both of them are common technologies within cloud-based environments and you can create a “serverless microservice”
 - by writing the code for a microservice and then setting it up to run as a serverless function
 - any microservice that needs to run only occasionally is a good candidate for a serverless microservice deployment model
- Although both technologies offer important advantages for cloud-native computing, they solve different sorts of problems and they do not always go hand-in-hand
 - Serverless is one way to host microservices, but it is not the only way
 - Nor you deploy and manage microservices in the same way as serverless functions



The entire path



Where it is convenient to use Docker



In the age of cloud, we are like “spoiled” children: *it is never enough*

- We expect software to meet business needs by performing (large and small) tasks quickly and inexpensively
- We expect to be able to immediately ramp up capacity on-demand and to tear it down immediately once done with it
- We do not care where the program runs. We just want it to run for how long we want
- We want changes to the software to be written and applied quickly

→ Virtual machines are not enough

→ We need and want autonomous management



Kubernetes

(Autonomous orchestration)



Motivation

- The real power of containers became clear when **orchestration** became available for it
- Google spawns more than *2 billion containers per week* with these systems
- The *standard de facto* for orchestration is **Kubernetes** that allows developers to configure containers and make them run the way you want them to run, and on which systems
- **Kubernetes**
 - allocates and manages storage used by containers
 - checks the health of containers and restarts or kills off unresponsive containers



Kubernetes

- **Kubernetes** = Greek for “pilot” or “helmsman of a ship”
- A **Container Orchestration System** based on *Borg* and *Omega* systems proven to work at Google for over 10 years. Created initially by three Google employees during the summer of 2014, improved and donated to an open source software foundation



kubernetes



Kubernetes is as the *Linux* for distributed systems

- In the same manner an OS abstracts away the hardware differences (with different CPU types, etc.), Kubernetes abstracts away the fact that you have thousands of nodes in the node pool and provides consistent operation methods for apps
- Manage the components of your applications like a **cattle** (generic, bulk operations) instead of like **pets** (every operation is customized with care and love for the individual)
- Kubernetes is a platform for building platforms as API service extensions
- The system administrator declares the **desired state**, and Kubernetes' main task is to **make the desired state the actual state**



The core primitive: A Pod

- The **Pod** is the basic, **atomically deployable unit** in Kubernetes that consists of one or many *co-located containers*
- A Pod represents a ***single instance of an application***
- The containers in a Pod share the loopback interface (localhost) and can share mounted directories
- Each Pod has its own, uniquely assigned and internal IP
- **Pods are mortal**, which means that if the node on which the Pod runs becomes unavailable, the workload also goes unavailable



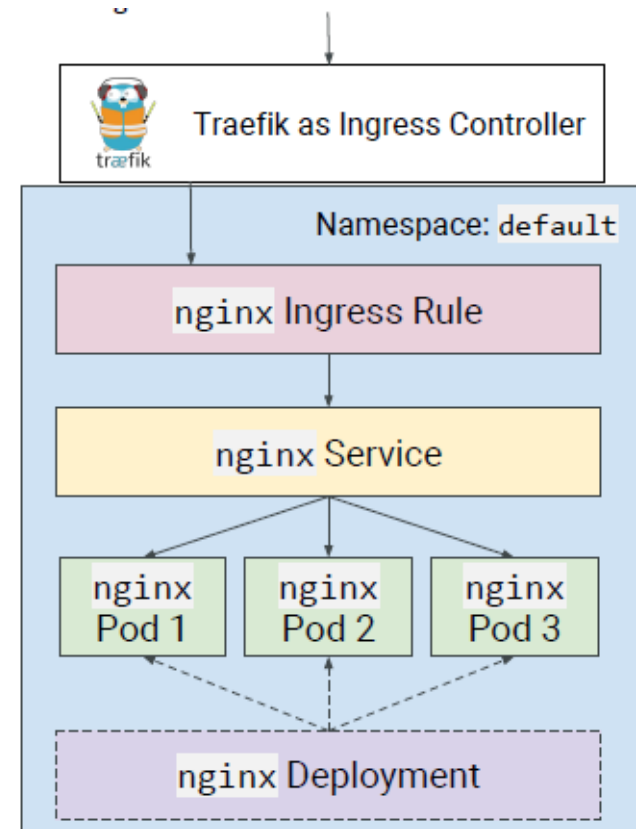
Exposing Pods

- A Service exposes one or many Pods via a ***stable, immortal, internal IP address***
- It is also accessible via cluster-internal DNS:
`{service}.{namespace}.svc.cluster.local`, e.g.
`nginx.default.svc.cluster.local`
- The Service selects Pods based on the **label key-value selectors**
- A Service may expose multiple ports. This ClusterIP can be declaratively specified, or dynamically allocated



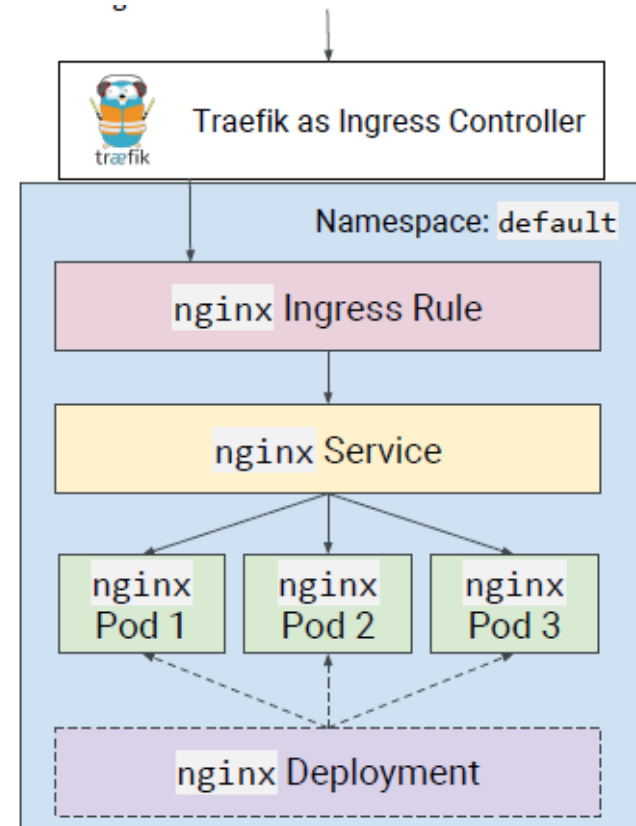
Virtual cluster

- A Service is only **accessible inside of the cluster**
- In order to expose the Service to the Internet, you must deploy an **Ingress controller** (e.g., Traefik) and create an Ingress Rule
- The Ingress rule is the Kubernetes-way of mapping hostnames and paths from *internet requests to cluster-internal Services*
- The Ingress controller is a load balancer that is creating forwarding rules based on the Ingress Rules in the Kubernetes API



Isolate all in a *Namespace*

- A Namespace is a logical isolation method, most resources are namespace-scoped
- You can group logically similar workloads in one namespace and enforce different policies
- You can have one namespace per team, and let them play in their own virtual environment
- Role Based Access Control (RBAC) can be used to control what Kubernetes users can do, and what resources in what namespaces a user can access is one of the parameters to play with

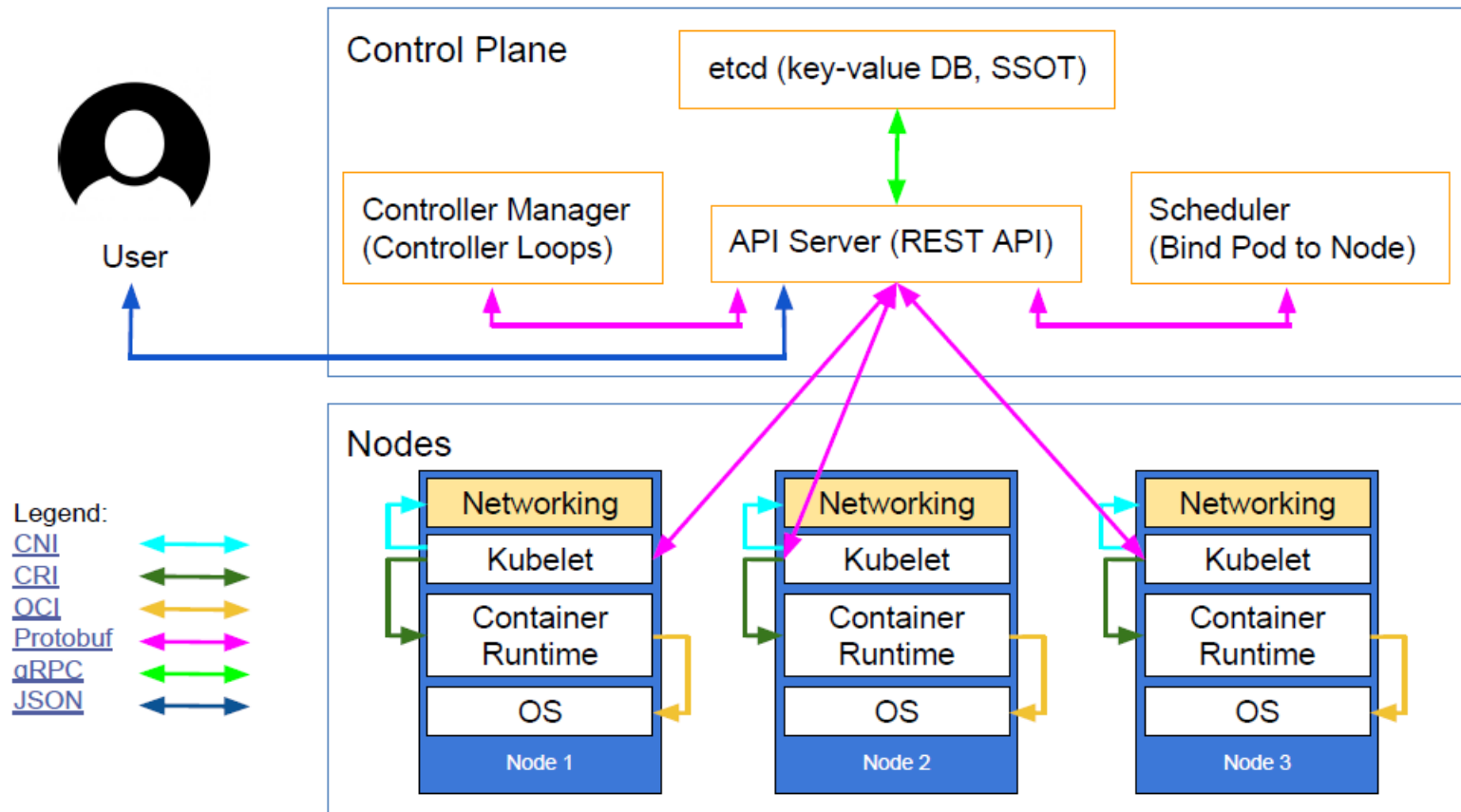


Deployment: a replicated, upgradeable set of Pods

- With a Deployment, you can manage Pods in a ***declarative and upgradeable*** manner
- Note the *replicas* field → Kubernetes will guarantee that amount of Pods created from the template are always available
- When the Deployment is updated, Kubernetes will perform a **rolling update** of the Pods running in the cluster.
- Kubernetes will create one new Pod and remove an old one until all Pods are new



Kubernetes architecture



Kubeadm

